A.D.A.P.T.: A Weather-Informed, Verifiable Pipeline from Annotated Flood Maps to Executable UAV Relief Missions

Abstract

Flood relief missions require an operator to identify delivery points, avoid hazards, transform map pixels into coordinates and encode autopilot commands. This paper evaluates A.D.A.P.T., a pipeline that converts flood-annotated map rasters and weather into QGroundControl WPL 110 missions. Operator-sampled HSV segmentation and a terrain-free spread rule produce obstacles; nearest-neighbour ordering and per-leg D* Lite produce routes. A second mode selects up to K bases and plans battery- and payload-limited sorties with a geometric crossing check. The evaluation uses three demonstration maps without georeferencing and deterministic operator samples from the pushed `research-validation` commit. Its independent WPL validator accepts all recorded missions; 98 789 representable coordinate round trips agree within 6.8×10^{-8} pixels, and tests detect 31 of 33 injected defects. Under moderate weather, increasing K from one to four raises served drops from 10 to 18 of 22 for Varanasi and from 17 to 34 of 41 for Kanpur. However, forecast obstacles cause all 14 single-UAV no-path legs, one thematic map is misclassified as 83% flood, and workload is unbalanced. The maps lack ground truth and georeferencing, and no ground-station or SITL execution has been recorded. The results establish software-level behaviour for the evaluated commit, not geographic accuracy or flight readiness.

Keywords: UAV relief, flood mapping, mission planning, multi-UAV allocation, geospatial verification, WPL 110.

1 Introduction

Floods interrupt roads when isolated communities need supplies. UAVs can map affected areas and carry small urgent payloads [1,2]. Flood extents are available in satellite and aerial products [3, 4], yet a relief mission still requires an operator to identify drops, order visits, avoid hazards, convert image locations to coordinates, and encode autopilot commands. The reviewed segmentation, routing and inundation studies usually stop at one of these stages [5–7]. Mission generation and its geographic correctness deserve evaluation as part of the chain.

1.1 Motivation and gap

A.D.A.P.T. accepts an annotated flood-map raster, weather and a geographic reference. Its two modes turn detected regions into QGroundControl WPL 110 mission files. Existing perception methods often end at a mask, while routing studies usually assume known demand locations. The capability matrix in Table 1 shows the narrower gap addressed here: an image-to-mission chain with independently checked coordinate and mission layers. The reviewed set is targeted, so it cannot establish an absolute priority claim.

1.2 Contributions

The paper makes three checkable contributions. First, it specifies and evaluates the committed pipeline from operator-sampled flood pixels through routing to mission items, including straight fallback legs when a grid path fails (Sections 3–4). Second, it evaluates the pushed multi-base planner under battery, payload and geometric crossing constraints, comparing $K = 1$ with up to four bases on identical map inputs (Section 4.13). Third, it separates software correctness from geographic and operational validity using an independent WGS84 oracle, a specification-based WPL validator, adversarial tests and recorded end-to-end results (Section 5). These are integration and verification claims; no optimality or flight-readiness claim follows.

The remainder reviews related work, formalises both planning modes, describes the method and verification, reports experiments, and closes with limits and next steps.

2 Related Work

2.1 UAVs in Disaster Response

Surveys of UAV use in disasters converge on three roles: rapid mapping and damage assessment, communication support, and delivery of small, time-critical payloads [1,2,8]. Erdelj *et al.* [1] frame these roles across the prediction, assessment and response phases, and humanitarian-logistics reviews confirm that delivery to areas cut off by floods is a recurring motivation [9]. Most of this literature is either conceptual or concerned with networking and coordination; for example, multi-UAV search-and-rescue frameworks address task allocation among vehicles [10], and recent flood-management work simulates UAV-cluster deployment and resource scheduling with edge computing [11]. Gioia *et al.* [12] review adaptive mission planning and hazard monitoring for wildfires, chemical releases and floods, and identify the integration of hazard-evolution models with mission planning as fragmented across domains. These works motivate the problem but do not provide a path from a flood map to a mission file.

2.2 Flood Mapping and Segmentation

Classical water mapping relies on spectral indices such as the NDWI [13] and, for all-weather operation, on synthetic aperture radar with automated thresholding chains [3]. Learning-based methods require labelled data; Sen1Floods11 provides a georeferenced benchmark for Sentinel-1 [4], and FloodNet provides pixel-labelled post-flood UAV imagery [14]. Convolutional networks have been applied to UAV flood-extent mapping [5]. Training-free colour-based methods remain attractive where labels are unavailable: Simantiris and Panagiotakis [15] combine LAB-colour masks, a vegetation index and edges for unsupervised flood segmentation of UAV photographs, and Ibrahim *et al.* [16] compare RGB/HSI clustering and region growing. All of these methods terminate at a flood mask; none produces delivery points or a flight plan. A.D.A.P.T. addresses a narrower perception problem than these works: it extracts flood pixels from *already annotated* map products by operator-sampled HSV thresholding [17], followed by mathematical morphology [18] and border-following contour extraction [19].

2.3 Flood Modelling and Prediction

Physically based and simplified inundation models range from raster storage-cell schemes such as LISFLOOD-FP [20] to cellular-automata (CA) models that replace the shallow-water equations by local transition rules on a digital elevation model [21–23]. CA models trade some accuracy for large speed-ups, but still require terrain data, rainfall or inflow forcing and calibration; Jamali *et al.* [23] also report weaker performance where flow momentum matters. Teng *et al.* [6] review this spectrum and its uncertainties. No reviewed model is coupled to UAV mission planning. The spread rule in A.D.A.P.T. is not a hydrological model: it is a terrain-free morphological proxy that

dilates and shifts the observed mask according to precipitation and wind, and it is used only to inflate obstacles.

2.4 Path Planning

Grid search with A* [24] is optimal for static maps. Incremental methods, D* [25] and its simpler successor D* Lite [26, 27], reuse search effort when edge costs change, which is valuable when a vehicle discovers obstacles during execution. Reviews of UAV path planning [28] catalogue classical, sampling-based and learning-based planners; they assume that the obstacle representation is given. A.D.A.P.T. uses D* Lite on a downsampled occupancy grid derived from current and predicted flood masks; as shown in Section 7, its present offline use does not exercise the incremental property.

2.5 Route Optimisation and Relief Logistics

Visiting a set of delivery points is a travelling-salesman-type problem. Exact dynamic programming is exponential [29], and the nearest-neighbour construction heuristic can be a logarithmic factor worse than optimal in the worst case [30]. Operations-research work on UAV delivery formulates truck–drone tandems [31] and drone fleets for last-mile relief [32]; Otto *et al.* [7] survey this area. Most closely related, Dong *et al.* [33] optimise multi-vehicle, multi-UAV flood-rescue delivery with flood-depth-dependent road speeds using mixed-integer programming and adaptive large-neighbourhood search. These models are more rigorous than A.D.A.P.T.'s ordering heuristic, but take demand locations and network states as given inputs rather than deriving them from imagery, and output schedules rather than autopilot missions.

2.6 Landing and Drop-Site Selection

Image-based selection of safe landing zones has been studied using multichannel imagery fused with map data [34] and texture-based flat-surface detection [35]. These methods score candidate sites for landing safety. A.D.A.P.T. has no comparable safety model: its "safe" drop point is a convex-hull vertex on the boundary of a flood region, chosen for accessibility from the route, without terrain, population or obstacle information.

2.7 Mission Generation and Geospatial Conversion

MAVLink is the de facto protocol between ground stations and open autopilots [36, 37]; missions are commonly exchanged as QGroundControl WPL plain-text files [38] whose command semantics are defined by the common message set [39] and refined per autopilot [40,41]. Converting image coordinates to geodetic coordinates requires a projection model [42]; web-map imagery uses Web Mercator, whose scale varies with latitude [43], and accurate geodesics on the ellipsoid are available from Vincenty's and Karney's algorithms [44, 45]. We found no reviewed work that generates WPL missions from flood imagery and verifies the geodetic and mission layers.

2.8 Synthesis and Gap

Table 1 summarises the capabilities reported by representative work. Perception, hazard modelling, path planning, route optimisation and site selection are each well developed, but in the reviewed literature they are rarely combined into a chain that starts from flood imagery and ends in an executable autopilot mission, and the correctness of the image-to-geodetic and mission-encoding steps is not reported. A.D.A.P.T. addresses this intersection with deliberately simple components. Its contribution is integration and verification; each of its components is weaker than the specialised state of the art in its own area.

3 Problem Formulation and Architecture

3.1 Notation

Let I be the input raster of $W_0 \times H_0$ pixels. If W_0 exceeds a display width D (750 px by default), the image is resized to I' of size $W' \times H'$ with $W' = D$, $H' = \mathrm{round}(H_0 s)$ and $s = D/W_0$; otherwise $I' = I$ and $s = 1$. All perception and planning operate on the pixel domain $\Omega = \{0, \ldots, W'-1\} \times \{0, \ldots, H'-1\}$ of I', with OpenCV conventions: x is the column (increasing to the east of a north-up map) and y the row (increasing to the south), and integer coordinates denote pixel centres.

The principal symbols and outputs are in Table 2.

3.2 Problem Statement

Given $(I, w, \varphi_0, \lambda_0, \rho)$, produce $\mathcal{W}$ such that the vehicle takes off at h, visits every d_i once at cruise altitude, descends and releases a payload at each d_i, avoids O wherever a free path exists on the planning grid, and returns to h. A.D.A.P.T. does not solve this as a joint optimisation problem. It decomposes it into a fixed sequence of heuristic stages (Fig. 1), each of which is specified in Section 4. No optimality is claimed for the composition. In particular, visit ordering ignores obstacles, and path planning is optimal only per leg and on a coarse grid.

Multi-UAV variant. With up to K vehicles, the task becomes: choose at most K flood-clear launch bases, assign each drop point to at most one base, and produce one mission per base. Each mission consists of closed sorties from its base that respect a battery reserve and a payload capacity, and no two missions' routes may cross. Drops that no base can serve are to be reported, not flown to. A.D.A.P.T. again uses heuristics: the fewest bases that cover the reachable drops, a nearest-feasible assignment and independent per-base planning (Section 4.13). It does not minimise mission time, balance workload or coordinate vehicles in flight.

3.3 Architecture and Interfaces

Fig. 1 shows the implemented data flow of the default single-UAV mode. The entry point validates its arguments and the geographic configuration before any work is done (Section 4.10), then runs the stages in the order shown. The only interactive step is colour sampling, in which the operator clicks on flood pixels. With the option `--mode multi`, the stages after obstacle modelling are replaced by the multi-base planner of Section 4.13 (Fig. 2); both modes share segmentation, weather, spread rule, obstacles, geodetic conversion and mission encoding. In the single-UAV mode, the planner and the geodetic conversion are the only stages with explicit failure semantics:

- a leg with no free path is logged and flown as a straight segment;
- a coordinate outside the conversion's validity domain aborts mission generation without writing a file.

A mission file left by an earlier run is removed once the input image has been loaded, so that a run which fails afterwards cannot leave an older mission that could be mistaken for its output. The multi-base mode instead writes every run into a new session folder and exits with status 2, writing no mission, when its plan is infeasible. The implementation is written in Python using OpenCV [46] and NumPy, and is accompanied by an offline verification layer (Section 5).

4 Proposed Method

This section specifies each stage exactly as implemented in the evaluated version of A.D.A.P.T.. Parameter values are the defaults used in all experiments (Table 5).

Table 1: Capabilities reported by representative related work and by A.D.A.P.T.. ✓: addressed; (✓): partially or under strong simplification; –: not addressed in the cited work as reported. The table reflects the stated scope of each paper and is not exhaustive.

Work (group)	Flood perception	Hazard evolution	Weather input	Obstacle-aware path planning	Delivery-point selection	Visit ordering	Executable autopilot mission	Geo/mission verification
Flood segmentation [5, 14, 15]	✓	–	–	–	–	–	–	–
SAR flood mapping [3, 4]	✓	–	–	–	–	–	–	–
Inundation models [20–23]	–	✓	(✓)	–	–	–	–	–
Incremental grid planning [25, 27]	–	–	–	✓	–	–	–	–
Relief / drone routing [31, 32]	–	–	–	–	–	✓	–	–
Flood-rescue collaborative delivery [33]	–	–	–	(✓)	–	✓	–	–
Safe landing-zone detection [34, 35]	(✓)	–	–	–	(✓)	–	–	–
Multi-UAV SAR / flood deployment [10, 11]	–	–	–	(✓)	–	✓	–	–
A.D.A.P.T. (this work)	(✓)[a]	(✓)[b]	✓[c]	✓	(✓)[d]	(✓)[e]	✓	✓[f]

[a] Annotated map products only, operator-sampled colour thresholds. [b] Terrain-free morphological proxy, uncalibrated. [c] Current precipitation and wind only. [d] Boundary geometry only, no safety model. [e] Nearest-neighbour heuristic; the optional multi-base mode assigns drops to up to K bases by a nearest-feasible rule under a battery model. [f] Software verification only; no ground-station, SITL or flight validation.

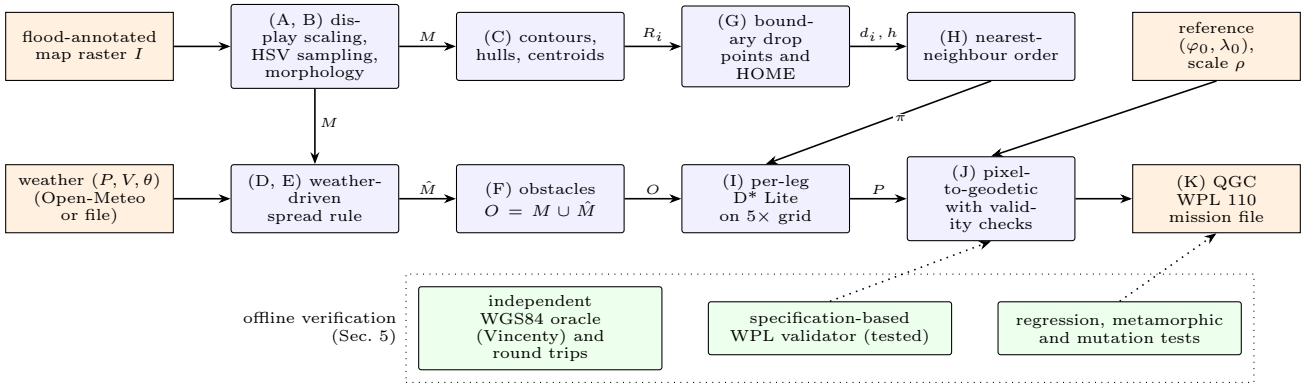


Figure 1: Architecture of A.D.A.P.T. as implemented. Letters refer to the subsections of Section 4. Blue boxes are pipeline stages executed in this order by the entry point; orange boxes are inputs and the output. The dotted block is the offline verification layer, which is not part of the run-time pipeline. Colour sampling is the only interactive step.

Table 2: Symbols and parameters used by both planning modes.

Symbol	Meaning
I, I'	Input raster and resized display image; $s = D/W_0$ is their scale.
$M, \hat{M}, O$	Detected flood, predicted flood, and planning obstacles $M \cup \hat{M}$.
P, V, θ	Precipitation, wind speed, and meteorological wind direction.
R_i, d_i, h	Flood region, assigned drop point, and HOME or launch base.
G, π, P_r	Occupancy grid, visit order, and pixel route.
K, E, r	Base cap, loaded endurance, and reserved battery fraction.
$\varphi_0, \lambda_0, \rho$	Centre latitude/longitude and metres per original image pixel.
$\mathcal{W}$	WPL 110 mission item sequence.

4.1 Input Acquisition and Display Scaling

The input is a raster in which flooded areas have been drawn in a distinct colour (red in all evaluated maps). The image is decoded with OpenCV and, if wider than D, downscaled with area interpolation to I' (Section 3). Scaling serves interactive display; because every later stage works in display pixels, all pixel-unit parameters below have a ground size that depends on s. This is accounted for in the geodetic conversion (Section 4.10) but not in the perception and planning parameters (Section 8).

4.2 Colour Sampling and Flood Segmentation

The operator clicks K flood pixels $c_k = (x_k, y_k)$ in I'. For each click, all pixels of the 11×11 patch Π_k centred on c_k (clipped at the image border) are converted to HSV [17] with OpenCV's ranges $H \in [0, 179]$, $S, V \in [0, 255]$. Let $\mathcal{S} = \bigcup_k \mathrm{HSV}(I'(\Pi_k))$. Writing C^- and C^+ for the minimum and maximum of channel C over $\mathcal{S}$, the thresholds are the sample extrema widened by margins $\delta_H = 20$ and $\delta_{SV} = 30$:

$$\ell_H = \max(H^- - \delta_H, 0), \quad u_H = \min(H^+ + \delta_H, 179), \tag{1}$$

$$\ell_C = \max(C^- - \delta_{SV}, 0), \quad u_C = \min(C^+ + \delta_{SV}, 255), \tag{2}$$

for $C \in \{S, V\}$. Because red straddles the hue origin, a *red-wrap* rule applies when $\ell_H \le 10$ or $u_H \ge 170$: the hue interval is then replaced by the fixed set $\mathcal{H} = [0, 10] \cup [170, 179]$ while the S and V intervals are kept; otherwise $\mathcal{H} = [\ell_H, u_H]$. The raw mask is

$$M_0 = \{p \in \Omega : H(p) \in \mathcal{H}, \; S(p) \in [\ell_S, u_S], \; V(p) \in [\ell_V, u_V]\}, \tag{3}$$

and the flood mask is obtained by a morphological closing followed by an opening [18] with a 5×5 square structuring element B:

$$M = (M_0 \bullet B) \circ B. \tag{4}$$

Two properties follow directly from these definitions and matter for the results. First, whenever the red-wrap rule fires, as it does for every map evaluated here, δ_H has no effect on M. Second, the patch contains background pixels around small or thin markings, so ℓ_S can fall to 0; since OpenCV assigns hue 0 to achromatic (grey and white) pixels, white background can then satisfy all three conditions. Section 7 shows both effects.

4.3 Flood-Region Extraction

External contours of M are extracted by border following [19], and contours with area below $a_{\min} = 200$ px^2 are discarded. Each remaining region R_i is represented by the vertex set of its convex hull and by its image-moment centroid $c_i = (\lfloor m_{10}/m_{00} \rfloor, \lfloor m_{01}/m_{00} \rfloor)$. Regions are taken from the current mask M only; predicted flooding affects obstacles but creates no new delivery targets.

4.4 Weather Acquisition

Current conditions at (φ_0, λ_0) are requested from the Open-Meteo forecast API [47]: precipitation, 10-m wind speed, 10-m wind direction and visibility (the last is not used). Responses are cached per coordinate, and requests time out after 10 s. On any request or parsing failure a calm fallback $(P, V, \theta) = (0, 0, 0)$ is returned and a warning is logged. Note that Open-Meteo reports current-condition precipitation as a sum over the preceding reporting interval, for example 15 minutes [48], whereas the spread rule below interprets P per hour. Wind above 40 km/h and precipitation above 15 mm trigger warnings but do not stop the pipeline. For reproducibility, the experiments use a weather file with $(P, V, \theta) = (10\ \text{mm}, 30\ \text{km/h}, 270^\circ)$.

4.5 Weather-Driven Spread Rule

A.D.A.P.T. estimates near-term flood growth with a deterministic, terrain-free rule applied to the binary mask (The spread rule). It can be read as a cellular automaton whose cells become flooded if any cell within an elliptical neighbourhood is flooded (isotropic growth), combined with a rigid advection of the flooded set in the downwind direction. With

$$b = \max(0, \lfloor P/5 \rfloor), \quad v = \max(0, \lfloor V/10 \rfloor), \quad \alpha = \theta + 180^\circ, \tag{5}$$

the shift is $(\Delta x, \Delta y) = (\text{round}(v \sin\alpha), \text{round}(-v \cos\alpha))$ display pixels, and $N = \max(1, \lfloor T \rfloor)$ iterations are applied. For the experimental weather, $b = 2$, $v = 3$, $\alpha = 90^\circ$ (eastward), $(\Delta x, \Delta y) = (3, 0)$ and $N = 2$ for $T = 2$ h. The rule has not been calibrated against observed flood evolution or compared with hydrodynamic models; its coefficients are documented in the code as heuristic. It is used only to make the planner keep clear of areas that may flood soon.

4.6 Obstacle Modelling

The obstacle set is $O = M \cup \hat{M}$, which equals $\hat{M}$ because the rule only adds pixels. Using O rather than M trades route exposure to predicted flooding against free-space connectivity; Section 7 quantifies both sides of this trade-off.

4.7 Drop-Point and HOME Selection

For each region, one hull vertex is chosen as its drop point. Let Γ be the polyline through the centroids $c_1, \ldots, c_n$ in contour-discovery order. Then

$$d_i = \arg \min_{q \in \mathcal{V}(\text{conv } R_i)} \min_j \text{dist}\left(q, \overline{c_j c_{j+1}}\right), \tag{6}$$

with the point-to-segment distance; for $n = 1$ the first hull vertex is used. The drop point is therefore on the flood boundary, not inside the water. It is "safe" only in this geometric sense: no terrain, structure, population or landing-safety information is used, unlike dedicated site-selection methods [34,35]. Γ joins centroids in discovery order, not in the final visiting order, and O is not consulted. HOME is the drop point nearest to the integer image centre: $h = \arg\min_i \|d_i - (\lfloor W'/2 \rfloor, \lfloor H'/2 \rfloor)\|$. The region whose drop point becomes HOME still receives a delivery.

4.8 Visit Ordering

The visiting order is built by the nearest-neighbour heuristic: starting at h, the closest unvisited drop point in Euclidean pixel distance is appended until all are visited, and h is appended at the end. Because h is itself a drop point, the first step has zero length. The procedure is $O(n^2)$, ignores obstacles, and carries no optimality guarantee; its worst-case ratio to the optimal tour grows logarithmically with n [30]. Section 7 measures its gap empirically.

4.9 Per-Leg Path Planning with D* Lite

Grid. O is downsampled by a factor $f = 5$ with nearest-neighbour resampling to an occupancy grid G of $\lfloor W'/5 \rfloor \times \lfloor H'/5 \rfloor$ cells. Each cell therefore reflects a single sample of its 5×5 block, so features thinner than a cell can be missed.

Endpoints. Consecutive stops (p, q) are mapped to cells by integer division and clamped to the grid. Because drop points lie on flood boundaries, endpoints frequently fall in occupied cells; such endpoints are moved to the nearest free cell by an 8-connected breadth-first search of radius 10 cells. If none is found, the endpoint stays inside the obstacle and a warning is logged.

Search. For each leg, a new D* Lite instance [26,27] is created on the 8-connected grid. The edge cost is $c(u, v) = \|u - v\|$ (1 or $\sqrt{2}$) if neither cell is occupied and ∞ otherwise, and the heuristic is the Chebyshev distance, which is admissible because every step costs at least one Chebyshev unit. With $\mu(u) = \min(g(u), rhs(u))$, vertices are ordered by the key

$$k(u) = \left[\mu(u) + h(s, u) + k_m;\ \mu(u)\right], \tag{7}$$

where s is the start cell and the search runs from the goal. Every occupied cell is registered through the standard vertex-update procedure before the shortest path is computed, and the path is extracted by repeatedly moving to the successor that minimises $c(u, v) + g(v)$. The implementation documents D* Lite as chosen for efficient replanning when obstacles are discovered. However, because each leg uses a fresh instance on a static grid, k_m remains 0 and no search effort is reused: *the incremental-replanning property of D* Lite is not exercised in the present offline pipeline.*

Failure behaviour. If the start cell has infinite cost-to-go, no intermediate points are added for that leg, so the mission flies a straight segment between the two stops, possibly across O; this is logged as an error. Interior path cells (g_x, g_y) are mapped back to display pixels as $(5g_x, 5g_y)$, the top-left corner of each block; the row actually sampled by the resampling can differ from it by up to two pixels.

4.10 Pixel-to-Geodetic Conversion

The geographic reference (φ_0, λ_0) is interpreted as the ground position of the geometric centre of the image, which in pixel-centre coordinates is $\mathbf{c} = ((W' - 1)/2, (H' - 1)/2)$. Since ρ refers to original pixels, the ground size of a display pixel is $k = \rho/s$. A route pixel (x, y) is converted by

$$E = (x - c_x)\, k, \qquad N = (c_y - y)\, k, \tag{8}$$

$$\varphi = \varphi_0 + \frac{N}{L}, \qquad \lambda = \lambda_0 + \frac{E}{L \cos\varphi_0}, \tag{9}$$

with $L = 111\,320$ m per degree. Equations (8)–(9) are an equirectangular projection with standard parallel φ_0 on a sphere of radius $R = 180L/\pi \approx 6\,378\,165$ m [42]. Relative to the WGS84 ellipsoid, the north–south scale is off by $|M(\varphi)\pi/(180L) - 1|$ and the east–west scale by $|N(\varphi)\pi/(180L) - 1|$, where M and N are the meridional and prime-vertical radii of curvature. At 25° latitude these are 0.49% and 0.06%, respectively, and second-order terms grow with the square of the offset.

The conversion refuses to return a coordinate, raising an error that aborts mission generation before any file is opened, when

- any input is non-finite, or $k \le 0$ (which would collapse or mirror the mission);
- $\varphi_0 \notin (-90^\circ, 90^\circ)$, where $\cos\varphi_0 = 0$, or $\lambda_0 \notin [-180^\circ, 180^\circ]$;
- the offset crosses a pole ($|\varphi| > 90^\circ$);
- $|\lambda - \lambda_0| \ge 180^\circ$, where wrapping would map distinct offsets to the same meridian, which also excludes overflowed offsets.

Table 3: Mission item sequence generated by A.D.A.P.T. (frame 3; altitudes in m relative to HOME, which is the base in the multi-base mode).

Item	MAVLink command (id)	Alt.	Parameters / role
0	NAV_WAYPOINT (16)	0	HOME, current flag 1
1	NAV_TAKEOFF (22)	100	at HOME
for each route point:			
	NAV_WAYPOINT (16)	100	cruise waypoint
additionally at each delivery stop:			
	NAV_LOITER_TIME (19)	100	p1 = 5 s
	NAV_WAYPOINT (16)	10	descend to drop altitude
	DO_SET_SERVO (183)	10	p1 = servo 9, p2 = 2000 μs
	NAV_WAYPOINT (16)	100	climb back
multi-base mode only, at each reload stop (Section 4.13):			
	NAV_LAND (21)	0	at the base
	NAV_TAKEOFF (22)	100	at the base
$m-1$	NAV_RETURN_TO_LAUNCH (20)	100	–
m	NAV_LAND (21)	0	at HOME

Otherwise λ is wrapped into $[-180°, 180°)$ only if it lies outside $[-180°, 180°]$. The same check is run on $(\varphi_0, \lambda_0, \rho)$ when the program starts, so invalid configurations fail before the interactive step.

The model further *assumes* that the image is north-up, has square pixels and uniform scale, and that (φ_0, λ_0) and ρ are known. None of these assumptions is checked against the image, and Section 7 shows that they do not hold for the supplied maps. Rounding H' makes the vertical display scale differ from s by less than 0.07% for the evaluated images.

4.11 Mission Encoding

The route is serialised as a QGroundControl WPL 110 file [38]: one tab-separated line per item with index, current flag, frame, command, four parameters, latitude, longitude, altitude and autocontinue. All positional items use frame 3 (global position, altitude relative to home) [39]. The item sequence is given in Table 3. For every stop other than the departure and the final return, the vehicle loiters, descends to the drop altitude, actuates a servo, and climbs back. All coordinates are converted before the file is opened, so a conversion failure leaves no partial file. According to the ArduCopter documentation, DO commands run when the preceding navigation command completes, so the servo command fires on reaching the drop altitude; TAKEOFF climbs from the vehicle's current location; and RTL should generally be the last command, so the trailing LAND item is not expected to execute on that autopilot [40]. Waypoint parameter 4 (yaw) is written as 0; ArduCopter ignores it, whereas PX4 interprets it as a north heading [41]. These semantics were established from documentation and have not been confirmed in simulation.

4.12 Complexity

With $A = W'H'$ display pixels, fixed-size morphology and contour extraction each scan $O(A)$ pixels per pass; N spread iterations cost $O(NA)$ for fixed kernels. Nearest-neighbour ordering of n drops costs $O(n^2)$. The grid has $V_g \approx A/25$ cells. Because a fresh D* Lite instance is constructed for every leg, the implementation cannot reuse earlier search work; one leg may explore the whole grid, so the search cost grows with both n and V_g. In the fleet mode, shortest-path fields are computed for candidate-to-drop distances, then up to K bases are scored and independent sorties are planned. Pairwise crossing checks grow quadratically in the total number of route segments. These are structural bounds, not measured scaling laws; only the recorded runtimes in Section 7 are empirical.

4.13 Approach B: Multi-Base Planning

Besides the single-UAV pipeline of Section 4 (Approach A), A.D.A.P.T. provides an optional *multi-base* mode, selected with the command-line option `--mode multi` (Approach B).

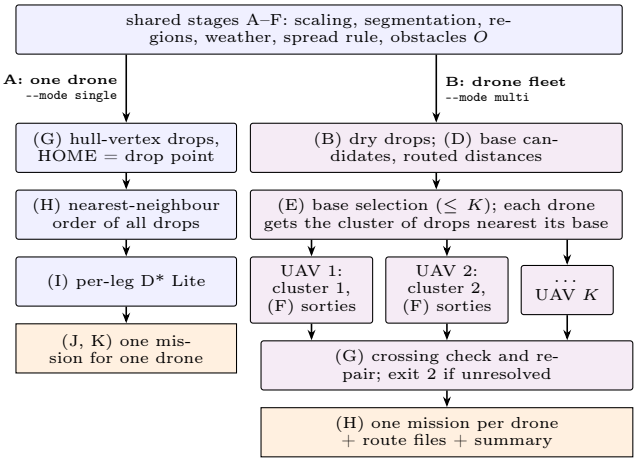


Figure 2: The two planning approaches. Left, Approach A: one drone visits every drop point (single-UAV pipeline, Section 4). Right, Approach B: a fleet of up to K drones (multi-base mode, `--max-bases K`; Section 4.13, whose subsections the letters refer to). Each drone flies battery-limited sorties from its own base to the cluster of drop points nearest that base. The drones are planned independently and coordinated only by the pre-flight crossing check; $K = 1$ yields one drone.

It selects up to K launch bases, assigns every drop point to at most one base and plans one mission per base; each base corresponds to one UAV. The result is that every drone serves its own cluster of nearby flood regions. The clusters are not computed by a separate clustering algorithm such as k-means. They follow from where the bases are placed (the base-selection step) and from assigning each drop point to the nearest base that can reach it. Section 7.5 measures how the regions are divided in practice. The mode shares stages A–F (scaling, segmentation, regions, weather, spread rule and obstacles) with the single-UAV pipeline and replaces stages G–K by the procedure below (Fig. 2). Its algorithms are those of a reference implementation developed in parallel with this work; Section 7.5 shows that the integrated code reproduces that implementation's plans exactly on identical inputs.

The mode is a *planner*, not a swarm system. The vehicles do not communicate, share state or react to each other in flight. Their coordination is limited to a pre-flight check that the planned routes do not cross (the crossing-repair step), and drops are assigned by a nearest-feasible rule without workload balancing.

4.13.1 Forecast Frames and Arrival-Time Obstacles

The planner checks obstacles at the time a vehicle arrives. The spread rule applies one iteration per hour, so its state after k iterations serves as the forecast frame for k hours: $F_0 = M$ and F_k is the output for horizon k, for $k = 1, \ldots, N$ with $N = \max(1, \lfloor T \rfloor)$. Frame k is stamped $t_k = \min(k, T)$, and $F_N = \hat{M}$. For a query time $t_a \leq t \leq t_b$ between consecutive frames, a pixel p is an obstacle if

$$(1-\alpha)\,\sigma_a(p) + \alpha\,\sigma_b(p) < 0, \qquad \alpha = \frac{t - t_a}{t_b - t_a}, \tag{10}$$

where σ is the signed Euclidean distance to a frame's flood boundary, negative inside. The flood front therefore advances linearly between frames. Times after the last frame use F_N. Base selection, connectivity and drop placement use $O = F_N$, the same obstacle set as the single-UAV mode.

4.13.2 Dry Drop Points

Hull vertices lie on M and would be rejected as flooded by the battery planner below, so this mode places drops on dry pixels next to each region. Let $\bar{R}_i$ be the filled region and $B_r = (\bar{R}_i \oplus Q_{2r+1}) \setminus O$ its free band of width r, where Q_{2r+1} is a square

element. With $r_{\max} = \max(2, \min(20, \lceil 0.03 \max(W', H') \rceil))$, let $r^* \le r_{\max}$ be the smallest r for which $B_r \neq \emptyset$. The drop is the point of B_{r^*} closest to the centroid polyline Γ, as in (6).

A region of area $A \ge 2500$ px^2 instead receives $n_i = \min(4,\ 2 + \lfloor (A - 2500)/3000 \rfloor)$ drops. For $j = 0, \ldots, n_i - 1$, the contour vertex at fraction j/n_i of the perimeter is an anchor, and its drop is the nearest point of $B_{r_{\max}}$ that lies at least 40 px from the region's earlier drops. An anchor without such a point gets no drop, and a region with no drop at all falls back to the single-point rule. All drops therefore lie outside O, within $r_{\max}$ display pixels of their region.

4.13.3 Battery and Payload Model

Battery use is modelled as a share of loaded endurance. A leg of length ℓ flown with payload m against a headwind w consumes

$$\beta(\ell, m, w) = \frac{\ell}{60\, v\, E} \left(1 + c_m m + c_w w\right) \tag{11}$$

of a full battery, with cruise speed $v = 5$ m/s, loaded endurance $E = 7$ min, $c_m = 0.08$ kg^{-1} and $c_w = 0.025$ s/m. Each drop carries $m_d = 0.25$ kg, the payload capacity is 8 kg, and a sortie must end with at least $r = 20\%$ of the battery left. The values are planning assumptions recorded in the configuration, derived from the published specification of a commercial 8-kg-payload multirotor; they have not been calibrated against any vehicle. The measured wind speed V is applied as a headwind $w = V/3.6$ m/s on every leg, the worst case for unknown headings. Only the site score of the base-selection step projects the wind onto each leg's heading. Reloading is assumed to be instantaneous.

4.13.4 Base Candidates and Routed Distances

Candidate sites are the pixels of a regular grid with stride $\lfloor \max(W', H')/40 \rfloor$ that lie outside O and at least 15 px from it, measured with a Euclidean distance transform. Distances are measured on a conservative *routing grid*. O is dilated by a 3×3 element, and a 5×5 block is blocked if any of its pixels is occupied. Unlike the single-UAV grid (Section 4.9), this grid cannot miss a flood strip that is thinner than a cell.

For every drop, an 8-connected shortest-path field with step costs 1 and $\sqrt{2}$ is computed. The routed distance $\delta(b, d)$ is the field value at the cell of b, multiplied by $5k$ metres; it is infinite if the two cells are not connected. Points are mapped to cells by a component-aware rule. A free cell is kept. A blocked cell is moved, by breadth-first search over at most 4 cells, to a free cell (of a required component, where one is given) whose straight connector to the original pixel crosses no pixel of O.

4.13.5 Base Selection and Assignment

A drop d is *reachable* from a site b if a single-drop sortie fits the usable battery, with β from (11):

$$\beta\big(\delta(b, d), m_d, w\big) + \beta\big(\delta(b, d), 0, w\big) \le 1 - r. \tag{12}$$

Drops that no candidate can reach are reported as unreachable. If one candidate reaches every remaining (*coverable*) drop, a single base is used. It is the reaching candidate with the lowest *score*: the flight time in hours plus the sum of sortie battery fractions of a greedy simulation that visits the drops in nearest-neighbour order by δ and closes a sortie before the reserve or the capacity would be exceeded. Otherwise, The base-selection procedure places up to K bases (`MAX_BASES`, default 4), at least 150 px apart.

Each drop is assigned to exactly one base: *the nearest base, by δ, that can reach it*. There is no balancing of workload between bases. Drops left unassigned are reported together with the reason: no site reaches them, the cap was reached, or every site that reaches them is too close to a chosen base. Setting $K = 1$ restricts the mode to a single base, that is, a single UAV.

4.13.6 Per-Base Mission Planning

Each base is planned independently with the single-UAV machinery and three additions.

1. *Connectivity.* Drops outside the base's free component of the routing grid are excluded and reported.

2. *Sorties.* Drops are ordered by the nearest-neighbour heuristic from the base and grouped into closed sorties. A drop joins the current sortie only if the whole sortie, including that drop and the return to the base, keeps the remaining battery at or above r and the payload within capacity. Otherwise the sortie is closed with a return and a reload, which resets the battery to full, and the drop starts the next sortie. The payload falls by m_d at each drop. A drop that a single-drop sortie from a full battery cannot serve, or that is flooded (by (10)) at its estimated arrival time, is reported as unreachable. Drops predicted to be flooded at their first arrival estimate are moved to the front of the order once.

3. *Routing.* Each leg is planned with D* Lite (Section 4.9) on the conservative grid of the obstacle mask at the leg's estimated arrival time. The arrival time is estimated as the routed length of the preceding legs plus the straight length of the current leg, divided by v. Endpoints are resolved with the component-aware rule above. A leg whose endpoints are not in one free component, or for which D* Lite returns no complete path, is flown as a straight segment and logged together with the number of obstacle pixels it crosses.

Sorties are first built from straight-line leg lengths. Routing and sortie construction then alternate, each pass replacing straight lengths by the routed lengths of legs already planned, until the stop sequence repeats or after at most $\min(6, n + 2)$ passes. Finally, battery use is recomputed along the routed geometry and logged for every sortie.

4.13.7 Route-Crossing Check and Repair

Because bases are planned independently, their routes can cross. All pairs of routes are tested for segment intersections, including collinear overlaps; the zero-length segments at reload stops are ignored. While a crossing remains, for at most five rounds, the first crossing pair is repaired with the first applicable tier:

0. *Reassignment.* If every drop of one crossing sortie is reachable from the other base, those drops are moved to it (the smaller move first), and both bases are re-planned.

1. *Rerouting.* The crossing leg of one route is re-planned with D* Lite, with the cells of the other route's crossing legs added as obstacles (first with a one-cell buffer, then without). The new leg is accepted only if it crosses no pixel of the arrival-time obstacle mask, keeps every sortie within the battery limit, reduces the number of crossings and creates none with a third base.

2. *Nudging.* A base is moved by up to 50 px in 5-px steps, nearest first, to a site that keeps the clearance and the separation and still reaches all its drops. It is re-planned, and the move is accepted if the new route crosses no other route.

If crossings remain, the run exits with status 2 and writes no mission. The check is purely geometric. All vehicles fly at the same cruise altitude, and no timing, vertical separation or minimum lateral distance between routes is enforced.

4.13.8 Output

After the check, one WPL 110 mission per base is written. It uses the encoding of Table 3, with a LAND and a TAKEOFF at the base for every reload. Each mission is validated at run time,

and a mission that fails is deleted. Each is also accompanied by a route file for the independent validator of Section 5, and a run summary records the assignment, the unreachable drops and their reasons, and the battery use of every sortie. Every run writes to a new session folder, so it cannot overwrite an earlier mission or be confused with one. As a diagnostic, a territory map assigns each grid cell to the base whose service points (the base and its visited drops) are nearest by routed distance; the run checks that every drop lies in its own base's territory.

4.13.9 Relation to the Single-UAV Mode

The two modes differ in more than the number of vehicles. The multi-base mode places drops on dry ground beside the flood rather than on its boundary. It launches from a flood-clear base rather than from a drop point, constrains sorties by battery and payload, and routes on a conservative, time-dependent grid. It also reports disconnected drops instead of flying straight across obstacles to them. Its $K = 1$ configuration is therefore the single-UAV baseline for multi-UAV comparisons (Section 7.5); it is not the single-UAV mode of Section 4.

5 Verification Methodology

5.1 Independent geodetic oracle

A test-only WGS84 oracle implements Vincenty's direct and inverse solutions without importing production code [44]. It is checked against a published worked geodesic to 1 mm and $0.01''$, an equatorial degree of longitude and the WGS84 meridian quadrant. A pixel's reference position is the point reached from the supplied centre by a geodesic of length $\sqrt{E^2 + N^2}$ at azimuth $\mathrm{atan2}(E, N)$. Model error is bounded using the ellipsoid's radii of curvature. Randomised forward/inverse round trips test valid and rejected inputs.

5.2 Mission validator

An independent specification-based validator reads each exported WPL file and checks item order, command parameters, coordinate ranges, drop actions and pixel-to-coordinate correspondence. It does not import production mission code. Thirty-six corrupted-mission cases exercise the validator itself. Passing it establishes format and declared-model consistency, not autopilot acceptance.

5.3 Metamorphic, mutation and fuzz tests

The same synthetic map is run through the entry point at nine display widths to check ground-position consistency. Mutation testing injects 33 plausible defects in a copy of the code, including axis and scale errors, servo-slot mistakes, a disabled crossing gate and broken reload bookkeeping. Seeded random grids, pathological end-to-end inputs and 500 multi-base scenarios check regression invariants. These tests use the pushed suite and recorded outputs.

5.4 SITL protocol and status

ArduCopter SITL should load an exported WPL mission and record item acceptance, servo-channel activation at drop altitude, RTL, intermediate LAND/TAKEOFF reload behaviour and battery response. A ground-station view, altitude and servo telemetry, and planned-versus-flown trajectory would be the evidence. The pushed repository contains no such run or telemetry; Section 7.9 reports its status without assuming success.

Table 4: Input data. All inputs are third-party demonstration images without georeferencing; "source" is the attribution printed on the image.

Map	Size (px)	Display	Content and printed source
Varanasi	972×970	750×748	red flood dots on web basemap; "ESA data ... processed by India Today"
Kanpur	976×966	750×742	red flood dots on web basemap; "Sentinel-2 SAR data ... ESA" (sic)
Assam	1048×739	750×529	thematic hazard map (5 classes, rivers, roads); no source printed

Table 5: Pipeline configuration used in all experiments (repository defaults). The multi-base experiment additionally uses a moderate weather preset (3 mm, 12 km/h, 270°).

Parameter	Value
Display width D; min. area $a_{\min}$	750 px; 200 px^2
HSV patch; δ_H, δ_{SV}; element B	11×11; 20, 30; 5×5
Weather (P, V, θ)	10 mm, 30 km/h, 270°
Horizon T; spread coefficients	2 h; 5 mm/px, 10 km/h/px
Grid factor; snap radius	5; 10 cells
Reference (φ_0, λ_0)	25.3176°, 82.9739°
Scale ρ	2.0 m/px
Cruise / drop altitude; loiter	100 / 10 m; 5 s
Servo channel; PWM	9; 2000 μs
Multi-base mode only (Section 4.13)	
Base cap K; separation; clearance	4; 150 px; 15 px
Speed v; endurance E; c_m; c_w	5 m/s; 7 min; 0.08/kg; 0.025 s/m
Payload per drop; capacity; reserve r	0.25 kg; 8 kg; 20%
Large region; extra drop per; max. drops	2500 px^2; 3000 px^2; 4

6 Experimental Setup

6.1 Data

No public benchmark with flood maps, delivery ground truth and georeferencing exists for the task of Section 3. The evaluation therefore uses the three flood-annotated maps distributed with the A.D.A.P.T. repository (Table 4). They are *demonstration inputs*, not a benchmark dataset. Two are news-style graphics in which detected flood pixels are drawn as red dots over a web-map basemap; the third is a thematic flood-hazard map with a legend. The PNG files contain no georeferencing: their only metadata are colour-profile chunks and a screen resolution of 5669 px/m. No manually annotated ground-truth masks exist, so segmentation accuracy (IoU, precision, recall) cannot be measured. The weather input is the repository's fixed weather file (Section 4.4); the live API was only smoke-tested. Synthetic images with known geometry are used for geospatial verification (Section 7.6).

6.2 Setup and Protocol

All experiments run the unmodified pipeline entry point on a laptop with an Intel Core i5-12500H CPU and 31.7 GiB RAM, under Windows 11 with Python 3.11.9, NumPy 2.4.6 and OpenCV 5.0.0, using the repository defaults (Table 5). Only three substitutions are made, all outside the pipeline code. Colour sampling is performed by a deterministic *simulated operator* that clicks the centroids of the $K = 8$ largest connected components of strongly saturated red pixels ($H \leq 5$ or $H \geq 175$; $S, V > 180$). Display windows are suppressed and their contents captured. The mission is written to the experiment folder instead of the default output path. All three maps use the repository's default geographic configuration, $(\varphi_0, \lambda_0) = (25.3176°, 82.9739°)$ and $\rho = 2.0$ m/px. Unless stated otherwise, results refer to the single-UAV mode; the multi-base mode is evaluated separately in Section 7.5. All scripts, raw results and figure generators are provided with the paper (Data Availability).

Table 6: End-to-end results (default configuration, simulated operator, fixed weather).

	Varanasi	Kanpur	Assam
Flood-mask fraction M (%)	8.10	15.08	83.10
Obstacle fraction O (%)	15.81	29.06	92.24
Regions = deliveries	16	33	1
Planned legs / no-path legs	16 / 8	33 / 6	0 / 0
Endpoints snapped / unsnappable	30 / 0	68 / 0	0 / 4
Route points	297	1010	3
Mission items	364	1145	10
Route length at configured scale (km)	5.66	18.78	0.00
Validator errors / warnings	0 / 0	0 / 0	0 / 0
Total runtime (s)	13.4	35.7	0.03
Share of runtime in planning	99.4%	99.6%	–

6.3 Metrics

We report the *flood-mask fraction* (share of display pixels in M) and the corresponding obstacle fraction for O; the number of regions, which equals the number of deliveries; planned legs and *no-path legs*, the latter flown as straight segments; endpoints moved off obstacles and endpoints that could not be moved (*unsnappable*); route points, mission items and runtime. For the ablation we use *route exposure*, the fraction of route length lying over a given pixel set, sampled every 0.5 px. For route ordering we use the ratio of the nearest-neighbour tour length to the optimum, and for geospatial verification the deviations in degrees, metres and pixels against the oracle.

7 Results and Discussion

7.1 End-to-End Results

Table 6 and Fig. 3 summarise the end-to-end runs. All three runs complete and produce missions that pass the validator with no errors or warnings. Path planning accounts for more than 99% of runtime; every other stage takes less than 0.1 s. Three findings qualify these outcomes.

1. *No-path legs.* Half of the Varanasi legs (8 of 16) and 6 of 33 Kanpur legs have no path on the planning grid. The mission therefore flies straight segments across obstacles on these legs (red segments in Fig. 3). The independent A* baseline (Section 7.4) confirms that these endpoints are genuinely disconnected on the grid: after snapping, they lie in free pockets enclosed by the inflated flood mask.

2. *Segmentation failure on the thematic map.* For Assam, the sampled patches include white background, ℓ_S falls to 0 and achromatic pixels pass the red hue test (Section 4-B). Of the image, 83.1% is labelled flood, the whole map becomes a single region, and both endpoints of the only (zero-length) leg remain inside obstacles. The resulting 10-item mission is structurally valid but meaningless.

3. *Geography.* All missions lie within about 2 km of the default reference, irrespective of the map content (Section 7.7).

7.2 Ablation: Predicted-Flood Obstacles

Table 7 compares planning on $O = M \cup \hat{M}$ with planning on M alone. Predicted-only pixels cover 7.7% (Varanasi) and 14.0% (Kanpur) of the image. Without predicted obstacles, 33.8% and 47.9% of the route length lies over predicted-only area. With them, exposure falls to 17.9% and 11.6%. Exposure does not reach zero because the no-path legs fly straight. The same inflation, however, causes *all* no-path legs: there are none with M alone, against 8 and 6 with O. It also increases planning time by a factor of 2.2–2.9, and for Varanasi it slightly increases exposure to the current flood (3.4% to 4.8%). The prediction stage is therefore not uniformly beneficial: in its current form, the benefit it is designed for comes with a loss of planner completeness.

Table 7: Ablation of predicted-flood obstacles. Exposure is the share of route length over the given area.

Map	Obstacles	No-path	Items	Time (s)	Exposure pred.-only	Exposure current
Varanasi	M	0	424	4.7	33.8%	3.4%
	$M \cup \hat{M}$	8	364	13.4	17.9%	4.8%
Kanpur	M	0	733	16.5	47.9%	5.5%
	$M \cup \hat{M}$	6	1145	36.4	11.6%	4.3%

Table 8: Sensitivity to the colour margin δ_{SV} and the number of clicks K ($\delta_H = 20$). Entries: flood-mask fraction (%) / regions / no-path legs. Hue margins of 10, 20 and 30 give identical results.

Setting	Varanasi	Kanpur	Assam
$\delta_{SV} = 15$, $K = 8$	7.2 / 19 / 8	14.0 / 32 / 6	83.1 / 1 / 0
$\delta_{SV} = 30$, $K = 8$ (default)	8.1 / 16 / 8	15.1 / 33 / 6	83.1 / 1 / 0
$\delta_{SV} = 45$, $K = 8$	8.8 / 18 / 10	15.2 / 32 / 6	83.1 / 1 / 0
$\delta_{SV} = 30$, $K = 4$	8.1 / 16 / 8	15.1 / 33 / 6	83.1 / 1 / 0
$\delta_{SV} = 30$, $K = 16$	9.7 / 23 / 8	15.1 / 33 / 6	83.1 / 1 / 0

7.3 Sensitivity

Table 8 varies the colour margins and the number of operator clicks. The hue margin $\delta_H \in \{10, 20, 30\}$ has *no* effect on any map, because the red-wrap rule replaces the hue interval. The saturation/value margin $\delta_{SV} \in \{15, 30, 45\}$ has a moderate effect on the two dot maps, while the Assam result is insensitive to every setting: its contamination is structural rather than a tuning artefact. Using 16 instead of 8 clicks raises the Varanasi flood fraction from 8.1% to 9.7% and the region count from 16 to 23, so the interactive step is a material source of variability. The ground scale ρ affects only the geodetic conversion: the Varanasi route length scales linearly at 2.83 km per m/px, so the 5.66 km mission at $\rho = 2$ m/px would be 263 km at the landmark-estimated scale of the map (Section 7.7).

7.4 Route Ordering and Planner Baseline

Table 9 reports two evaluation-only baselines; neither is part of A.D.A.P.T.. Against exact optima from Held–Karp dynamic programming [29] on uniformly random instances, the nearest-neighbour order is on average 4.6% (5 points) to 12.0% (12 points) longer, with a worst case of 44.5%. On the 16 Varanasi drop points it is 3.2% longer than optimal. For the 33 Kanpur points only a minimum-spanning-tree lower bound is available, which limits the gap to at most 37.8%.

The per-leg planner was compared with a plain grid A* [24] using the same grid, endpoints, costs and heuristic. The two agree on reachability for all 49 non-trivial legs and return equal path costs (difference below 2×10^{-13}), which supports the correctness of the D* Lite implementation. As used in the pipeline, however, D* Lite is 61 to 112 times slower than A*. Each leg builds a new instance and registers every obstacle cell, which amounts to 0.84 and 2.38 million vertex updates for the two maps, so its incremental design brings no benefit in this offline setting.

7.5 One Drone vs. a Drone Fleet (Single- vs. Multi-UAV Planning)

Protocol. The multi-base mode (Section 4.13) was run on the three maps with the simulated operator, under the dummy weather and under a moderate preset (3 mm, 12 km/h, 270°), with base caps $K \in \{1, 2, 3, 4\}$ (24 runs). For a given map and weather, all caps share the drop points, the obstacles and the vehicle model, so K is the only variable. $K = 1$ is the single-UAV case, in which one vehicle flies all sorties from one base. All metrics are read from each run's own outputs, and every exported mission was re-checked by the independent validator against its route file. The *estimated completion time* is the routed length of the longest mission divided by $v = 5$ m/s. It assumes that all vehicles launch together and excludes take-off, landing, drop manoeuvres and reload turnaround, so it is a

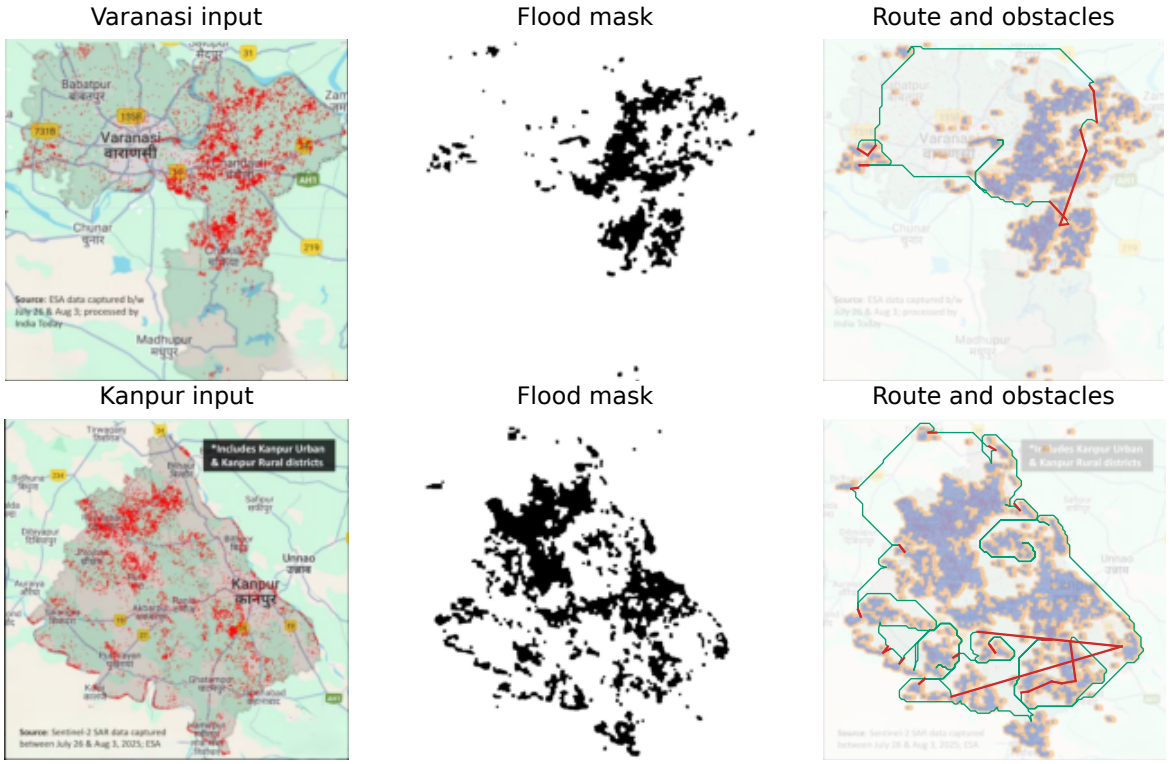


Figure 3: Recorded pipeline outputs for Varanasi and Kanpur: annotated input, extracted flood mask, and planned route over current (blue) and predicted-only (orange) obstacles. Red route segments mark long straight fallbacks; they are not validated flight paths. The third-party input imagery is reproduced for research review only.

Table 9: Evaluation-only baselines: route-ordering optimality and per-leg planner comparison.

Nearest neighbour / optimal tour length (random instances)			
Points n (instances)	Mean	Max	Optimal found
5 (300)	1.046	1.257	42.3%
8 (300)	1.085	1.380	18.0%
12 (100)	1.120	1.445	11.0%
Varanasi, $n = 16$		1.032 (exact optimum)	
Kanpur, $n = 33$		$\leq$1.378 (MST bound)	

Per-leg D Lite (as used) vs. grid A*, identical inputs*		
	Varanasi	Kanpur
Legs; reachability agreement	16; 16/16	33; 33/33
Max. |cost difference|	4×10^{-14}	1×10^{-13}
Runtime D* Lite / A* (s)	12.67 / 0.21	30.83 / 0.28

lower bound rather than a prediction of mission time. Table 10 gives the results; Fig. 4 plots coverage and completion time, and Fig. 5 reconstructs the Kanpur routes from the committed route files.

Feasibility and safety checks. Of the 24 runs, 20 produced missions, 43 in total. In these runs:

- all 43 missions pass the validator with no errors;
- no two routes of a run cross, so no repair was needed;
- no leg was flown as a straight fallback, and no route pixel lies in the detected flood;
- every visited drop lies in its own base's territory;
- the largest routed sortie uses 79.6% of the battery, against the 80% limit.

The four Assam runs under moderate weather end with status 2, because no base site reaches any drop. Under dummy weather, Assam has one base and visits 1 of 4 drops for every K. Both outcomes follow from the segmentation failure of Section 7.1.

Coverage. A base can only serve drops within one single-drop sortie (12). That is about 832 m of routed distance in calm air, 768 m under the moderate wind and 690 m under the dummy wind, or roughly 300 and 270 display pixels at the Varanasi scale. Adding bases therefore mainly adds coverage. From $K = 1$ to $K = 4$, the visited drops rise as follows:

- under dummy weather, from 6 to 12 of 22 (Varanasi) and from 10 to 23 of 41 (Kanpur);
- under moderate weather, from 10 to 18 and from 17 to 34.

Every unvisited drop is reported. In all but two runs, the reason is that no admissible base reaches it within the cap; in two Kanpur runs, the sortie planner rejected one assigned drop. The stronger dummy wind shortens the reach and lowers coverage at every K. Because the visited sets differ, total route length does not compare efficiency like for like. It grows with K because more drops are served and each base flies its own return legs.

Division of work into clusters. To see how Approach B divides the map, every drop point was attributed to its flood region in each run. The flood regions were recomputed exactly as in the run, and each drop point was matched to the nearest region contour. In the 12 runs with more than one drone, 190 of the 206 flood regions that received a delivery were served by a single drone, and each drone served a cluster of 2 to 18 neighbouring regions. The remaining 16 regions, all large regions with drop points on several sides, were shared by two drones. One drone per cluster is therefore what the planner produces in practice, although it is not imposed.

Workload. The nearest-feasible assignment does not balance work. On Kanpur under moderate weather, the drops per UAV are as follows:

- $K = 1$: 17;
- $K = 2$: 17 and 8;
- $K = 3$: 17, 7 and 6;
- $K = 4$: 15, 7, 6 and 6.

Table 10: One drone vs. a drone fleet: single- and multi-UAV planning with the multi-base mode on the same missions (simulated operator, `METERS_PER_PIXEL`= 2 m per original pixel). K is the base cap; $K = 1$ is one UAV flying every sortie from one base. Visited: drops delivered of dry drop points generated. km: routed length of all missions; max/UAV: longest single mission. Time: estimated completion (minutes) from route length at 5 m/s, all UAVs launched together, excluding take-off, landing, drop manoeuvres and reload turnaround. Every exported mission passed the independent validator; no two routes cross.

		Dummy weather (10 mm/h, 30 km/h)						Moderate weather (3 mm/h, 12 km/h)					
Map	K	UAVs	Visited	Sorties	km	max/UAV	Time	UAVs	Visited	Sorties	km	max/UAV	Time
Varanasi	1	1	6/22	3	3.0	3.0	10	1	10/22	8	10.3	10.3	34
	2	2	9/22	5	5.3	3.0	10	2	16/22	10	13.0	7.6	25
	3	3	11/22	6	6.3	2.3	8	3	18/22	10	12.9	6.4	21
	4	4	12/22	6	5.7	2.1	7	3	18/22	10	12.9	6.4	21
Kanpur	1	1	10/41	3	3.1	3.1	10	1	17/41	9	11.0	11.0	37
	2	2	16/41	5	5.4	3.1	10	2	25/41	14	17.2	11.0	37
	3	3	21/41	8	8.7	3.2	11	3	30/41	16	19.4	11.0	37
	4	4	23/41	9	9.4	3.2	11	4	34/41	16	19.6	8.5	28
Assam	1	1	1/4	1	0.5	0.5	2	infeasible: no base reaches any drop (exit 2)					
	2	1	1/4	1	0.5	0.5	2	infeasible: no base reaches any drop (exit 2)					
	3	1	1/4	1	0.5	0.5	2	infeasible: no base reaches any drop (exit 2)					
	4	1	1/4	1	0.5	0.5	2	infeasible: no base reaches any drop (exit 2)					

The longest mission therefore stays at 11.0 km, and the completion estimate at 37 min, up to $K = 3$; the estimate falls to 28 min only at $K = 4$. On Varanasi under moderate weather, the longest mission falls from 10.3 to 6.4 km and the completion estimate from 34 to 21 min, using three of the four permitted bases.

Choosing an approach. The planner chooses between one drone and a fleet by itself. If one base site reaches every reachable drop point within battery range, it returns a single base even when $K > 1$; Assam under dummy weather shows this. A fleet is therefore used only where one drone cannot cover the map, and there it served up to twice as many drops (Kanpur, moderate weather: 17 to 34). The single-UAV mode of Section 4 (Approach A) delivers to every region in one tour: 16 and 33 drops on Varanasi and Kanpur (Table 6). However, it has no endurance check. Its routes of 5.66 and 18.78 km at the configured scale exceed what one battery of the fleet mode's vehicle model allows (about 2.1 km at 5 m/s in 7 min). For a battery-limited vehicle, the one-drone baseline is therefore the $K = 1$ configuration of the multi-base mode.

Scale dependence. These coverage figures hold only for the configured scale. The $K = 4$, moderate-weather runs were repeated at the landmark-estimated scales of Section 7.7 (93 and 130 m/px). At those scales a single-drop sortie spans only 6.4 and 4.5 display pixels, every drop is unreachable, and both runs end with status 2.

Runtime. A feasible run took between about 10 s and 3 min on the evaluation laptop. These are single measurements, taken with several runs executing in parallel.

Equivalence with the reference implementation. To check that integrating the mode preserved its behaviour, both implementations were run on Varanasi and Kanpur with identical inputs:

- the same operator samples and weather;
- the same ground size of a display pixel, 2 m (the reference does not compensate for display scaling, so its scale was matched explicitly);
- for non-calm weather, the reference implementation's own forecast frames, because the reference uses a different spread rule.

Three cases were compared: calm weather on both maps, and a moderate-weather Varanasi case in which two reassignment repairs occur. In all three, the base positions, the assignment, every route point and every drop and reload index are identical. Assam could not be compared, because the reference rejects its flood mask (83% coverage) before planning.

7.6 Geospatial and Mission Verification

Table 11 summarises the verification results. Every result in this subsection concerns *software correctness*, that is, whether the code computes what its model specifies. None of it shows that the generated coordinates match real places.

- *Synthetic ground truth.* On a 1000 × 1000 px synthetic image at 10 m/px, the implementation matches an independent re-derivation of (8)–(9) to within 1.4×10^{-14} degrees at nine control points. Against the WGS84 geodesic ground truth, the largest error is 25.9 m at the image corners, dominated by the 0.49% north–south scale error of L at 25°, and every point is within the derived bound.
- *Round trips and edge cases.* Of 100 000 randomised round trips through a test-only inverse, covering mid-latitudes, near-polar and near-antimeridian references, image sizes and scales from 0.05 to 633 m/px, 98 789 are representable and agree within 6.8×10^{-8} px; the remaining 1 211 are correctly rejected.
- *Mission correspondence.* Every row of the three missions (1 519 rows in total) matches an independent conversion of its source pixel within 3.6×10^{-15} degrees. Traced back to the original image, the deviation is at most 0.57 m, entirely north–south and explained by the rounding of H'.
- *Resize consistency.* A synthetic three-region map run at nine display widths (300 to 2400 px, odd and even) yields drops at the same physical corners. They deviate from the model by at most one display pixel, and by 7×10^{-10} m when no resizing occurs.
- *Mutation testing.* 31 of 33 injected defects are detected by the 231-test suite. Of the 24 defects in the geospatial and mission layers, 22 are detected; all 9 defects injected into the multi-base integration are detected, for example an uncompensated display scale, a disabled crossing gate, a missing reload stop or a stale reach set after nudging. The two surviving mutants are behaviour-equivalent: one removes a redundant finiteness guard, the other repeats a conversion.
- *Adversarial testing.* A separate QA pass combined three kinds of test: property-based tests (for example, D* Lite against Dijkstra on random grids, and the crossing test against a brute-force check), about 110 end-to-end runs on pathological inputs, and 500 seeded random scenarios checked against the multi-base invariants. It found ten defects, all now corrected:
  - crashes on non-finite command-line values, on incomplete weather data and on invalid environment variables;
  - an excessive memory request for implausible precipitation values;
  - drops of a base without a route that were missing from the run summary;

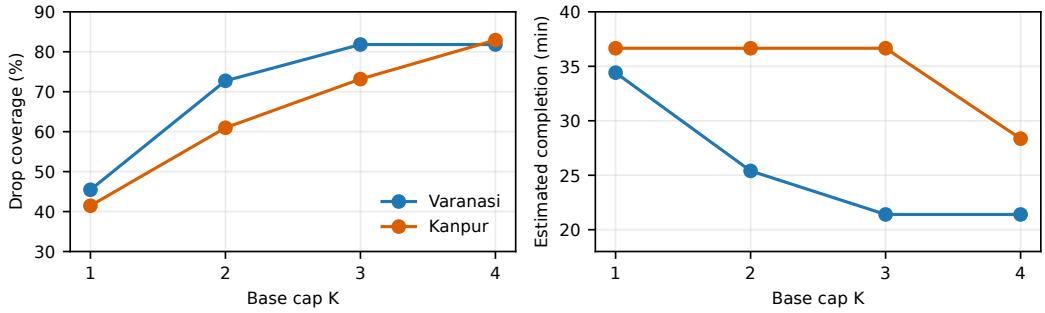


Figure 4: Moderate-weather fleet experiment from committed results. Drop coverage and optimistic completion time versus the allowed number of bases K; Varanasi uses three bases when $K = 4$. The completion estimate excludes takeoff, landing, drop and reload time.

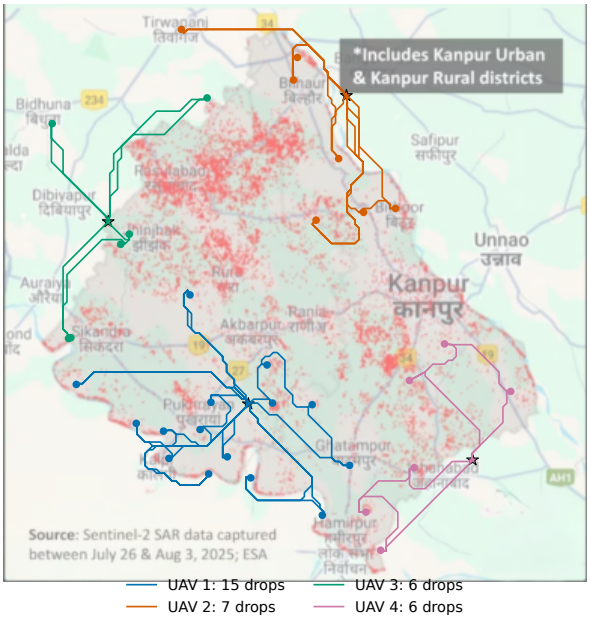


Figure 5: Kanpur, moderate weather, $K = 4$: four bases (stars), their served drops (dots), and planned routes. Reconstructed from committed route records; the map image has no georeference.

– coincident drop points that the repair step split between two bases.

None of the corrections changes a mission generated in the experiments above.

During verification, the procedure exposed and led to the correction of defects in the original implementation: scale was not compensated after resizing, servo parameters were in the wrong slots, the region chosen as HOME received no delivery, the reference pixel was off by half a pixel, and invalid or polar inputs silently produced invalid coordinates. All results reported here use the corrected code.

7.7 Geographic Validity of the Supplied Maps

To test whether the configured geography could describe the maps, a similarity transform was fitted between visible place-name labels and approximate public coordinates of those towns. This was done for 5, 10 and 12 labels on the Varanasi, Kanpur and Assam maps; the label positions were read off by eye. The estimate is a plausibility check, not a georeference. Residuals are 15–26 px and the rotations are $-0.5°$, $-1.0°$ and $-4.8°$. The fitted scales are about 93, 130 and 633 m/px, which is 46, 65 and 317 times the configured 2.0 m/px. The default

Table 11: Software verification of the geospatial and mission layers, and overall validation status.

Check	Result
Software correctness (performed)	
Automated tests (incl. adversarial)	231 + 520 subtests pass
Oracle vs. published geodesic	$\leq$1 mm, $\leq 0.01''$
Implementation vs. model (synthetic)	$\leq 1.4 \times 10^{-14}$ deg
Model vs. WGS84, $\leq$7.1 km offsets, 25°	$\leq$25.9 m
Round trips (98 789 compared)	max 6.8×10^{-8} px
Mission rows vs. source pixels (1 519)	$\leq 3.6 \times 10^{-15}$ deg
Resize, 9 display widths	$\leq$1 display px
Mutants detected	31/33 (2 equivalent)
Validator attack cases	36 pass
Single-UAV missions (3 maps)	0 errors, 0 warnings
Multi-base missions (20 runs, 43 files)	0 errors, no crossings
Multi-base plans vs. reference (3 cases)	identical
Fuzzed end-to-end scenarios (500)	0 invariant violations
Validation beyond software correctness	
Geographic validity of supplied maps	not established
Ground-station load test (Mission Planner, QGC)	not performed
Software-in-the-loop (ArduPilot/PX4 SITL)	not performed
Hardware-in-the-loop / physical flight	not performed

reference lies 23 km, 319 km and about 1 000 km from the estimated image centres. At those extents, the flat-earth model itself would err by up to 345 m, 563 m and 7.7 km (from the WGS84 oracle), and the maps' web-Mercator basemaps violate the uniform-scale assumption [43]. The generated missions are therefore *not* geographically meaningful for these maps. The cause lies in the inputs and configuration, not in the conversion code.

7.8 Mission Semantics

The command parameters were audited against the MAVLink definitions and the ArduPilot and PX4 documentation [39–41, 49]; the findings are given in Section 4-K and Section 8. The lower block of Table 11 separates what has and has not been validated. The reload stop of the multi-base mode, a LAND followed by a TAKEOFF at the base, was not covered by this audit. Whether an autopilot resumes the mission after an intermediate LAND depends on its configuration, for example disarming after landing, and must be established in simulation.

7.9 Simulation and Mission Execution

No simulation or ground-station execution has been performed for this manuscript (Table 11). Each exported mission (one file in the single-UAV mode, one file per base in the multi-base mode) is to be loaded in QGroundControl or Mission Planner and flown in ArduPilot SITL, with one simulated vehicle per base in the multi-base mode. The check should confirm that every item is accepted, that the servo actuates at each drop, how the reload LAND/TAKEOFF pair behaves, and how the vehicle executes RTL. Until it has been carried out, the operational-validity statements of Section 8 apply unchanged.

8 Limitations and Threats to Validity

Input validity. The three supplied PNG maps have no georeference or labelled flood truth. Their configured centre and scale do not match the depicted geography (Section 7.7), so exported coordinates cannot be used for field missions. Segmentation measures annotation colour, not water in raw imagery. The Assam thematic map is an observed failure: 83.1% of its pixels are classified as flood. Operator-click and weather variability were not sampled statistically. The colour-margin sweep changes the mask, but does not supply IoU, precision or recall.

Forecast validity. The spread rule is a terrain-free obstacle-inflation proxy, with no observed next-frame comparison. It ignores drainage and elevation and interprets current precipitation as an hourly quantity even though the API may report accumulation over a shorter interval [48]. Lower route exposure to its own predicted area is not evidence of flood forecast skill. No-prediction is the only tested forecast baseline.

Route and mission validity. The single-UAV mode uses one sampled cell per 5×5 block, can place drops in blocked cells, and flies a straight line when a leg is disconnected. Its 14 observed no-path legs therefore remain unsafe. It has no endurance check. Multi-base drops are dry under the program's mask, but have no independent landing or crowd-safety assessment. Multi-base battery coefficients are uncalibrated; wind is charged as a full headwind on every leg. The nearest-feasible assignment can leave Kanpur's longest mission at 11 km through $K = 3$. Crossing repair checks geometry only; it does not provide separation in time or altitude. Intermediate LAND/TAKEOFF reload behaviour is untested. These limitations also explain why standard multi-depot VRP and balancing baselines remain needed.

Study design. Results come from three convenience maps, one deterministic simulated operator, one single-UAV weather setting and two fleet weather settings. Runtimes are single measurements on one laptop, sometimes taken alongside parallel runs. The ground scale changes feasibility drastically: at the landmark-estimated Varanasi and Kanpur scales no drop is reachable. Neither flood-spread accuracy nor battery calibration can be inferred from these experiments. The related-work search was targeted rather than systematic.

Operational validation. The WPL validator checks format and model invariants, not acceptance or execution by an autopilot. No ground-station load, SITL, telemetry, hardware-in-the-loop or flight results are present in the pushed commit. Before any operational use, the exact missions require georeferenced inputs, coordinate inspection, SITL checks of servo timing, RTL and reload semantics, vehicle-specific failsafes, verified release hardware, a trained pilot and applicable aviation approval. Multiple vehicles additionally require temporal or vertical deconfliction.

9 Conclusion

This paper presented A.D.A.P.T., an open pipeline that converts a flood-annotated map raster and current weather into a QGroundControl WPL 110 UAV relief mission. It combines colour-based flood extraction, a weather-driven obstacle-inflation rule, boundary drop-point selection, nearest-neighbour ordering, per-leg D* Lite planning and a flat-earth geodetic conversion with explicit validity checks. It offers two approaches. Either one drone visits every drop point, or a fleet of drones is used, in which each drone serves the cluster of drop points around its own launch base under a battery and payload model. Fleet plans whose routes cross are rejected. The work addresses an intersection that the reviewed literature treats in separate pieces. Its contribution is integration and verification rather than any individual algorithm.

On three published flood maps the pipeline produced missions with 16, 33 and 1 deliveries, all of which passed an independent specification-based validator. The geospatial and mission layers were verified against an independent WGS84 oracle: mission rows match their source pixels within 3.6×10^{-15} degrees, 98 789 randomised round trips agree within 7×10^{-8} px, and 31 of 33 injected defects were detected. With identical inputs, the multi-base mode reproduced its reference implementation's plans exactly. In the fleet runs, 190 of 206 served flood regions went to a single drone, so each drone in effect took its own cluster. Raising the number of bases from one to four increased the drops served from 10 to 18 of 22 (Varanasi) and from 17 to 34 of 41 (Kanpur) under moderate weather. All 43 resulting missions passed the validator, and no routes crossed. Because the assignment does not balance work, however, one Kanpur base kept 15 to 17 drops throughout. The same evaluation exposed limits of the approach. Predicted-flood obstacles reduce the route's exposure to predicted flooding by 47% and 76%, but cause all 14 no-path legs, which are flown as straight segments. Segmentation of a thematic map labels 83% of it as flood. The nearest-neighbour order is up to 44.5% longer than optimal on random instances. The per-leg D* Lite is 61–112 times slower than A* because its incremental reuse is unused. Finally, the supplied maps are not georeferenced and their configured scale is wrong by factors of 46–317, so the generated coordinates are not geographically meaningful.

A.D.A.P.T. is therefore best understood as a verified software baseline for image-to-mission research, not as a deployable system. The most important next steps are ground-station and SITL validation of both modes, georeferenced inputs, segmentation ground truth, explicit handling of disconnected drop points in the single-UAV mode, and workload-aware, temporally deconflicted multi-UAV allocation.

Data Availability

The evaluated A.D.A.P.T. source code, test suite, evaluation scripts (`paper/experiments`), raw results (`paper/experiments/results`) are available at `https://github.com/TSaha4/A.D.A.P.T.-Autonomous-Drone-for-Adaptive-Path-Tracking/tree/research-validation` (pushed commit `fdb9cb3`; all reported measurements refer to this commit). The pushed `main` commit `ca27215` and `time-aware-routing` commit `607bc5f` diverge from it at `775165d`; their later code is not included in these measurements. The three input maps are third-party graphics reproduced from published sources. The authors must confirm redistribution rights before public release, or replace them with openly licensed georeferenced flood products. No ground-truth flood masks, georeferences or flight logs exist for these inputs.

Declarations

Code and data availability. The source, tests, evaluation scripts and recorded outputs are available at the repository and commit above. The supplied images are third-party graphics; redistribution rights require confirmation. No ground-truth flood masks, observed prediction pairs or flight logs accompany them.

Funding and competing interests. To be completed by the authors before submission.

Author contributions. To be completed by the authors before submission.

AI assistance. AI tools assisted manuscript drafting, evaluation scripts, verification tests and integration review. The authors remain responsible for checking every claim, result and disclosure before submission.

References

[1] Milan Erdelj, Enrico Natalizio, Kaushik R. Chowdhury, and Ian F. Akyildiz. Help from the sky: Leveraging uavs for disaster management. *IEEE Pervasive Computing*, 16(1):24–32, January 2017. doi: 10.1109/mprv.2017.11.

[2] Sharifah Mastura Syed Mohd Daud, Mohd Yusmiaidil Putera Mohd Yusof, Chong Chin Heo, Lay See

Khoo, Mansharan Kaur Chainchel Singh, Mohd Shah Mahmood, and Hapizah Nawawi. Applications of drone in disaster management: A scoping review. *Science & Justice*, 62(1):30–42, January 2022. doi: 10.1016/j.scijus.2021.11.002.

[3] André Twele, Wenxi Cao, Simon Plank, and Sandro Martinis. Sentinel-1-based flood mapping: a fully automated processing chain. *International Journal of Remote Sensing*, 37(13):2990–3004, June 2016. doi: 10.1080/01431161.2016.1192304.

[4] Derrick Bonafilia, Beth Tellman, Tyler Anderson, and Erica Issenberg. Sen1floods11: a georeferenced dataset to train and test deep learning flood algorithms for sentinel-1. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 835–845. IEEE, June 2020. doi: 10.1109/cvprw50498.2020.00113.

[5] Asmamaw Gebrehiwot, Leila Hashemi-Beni, Gary Thompson, Parisa Kordjamshidi, and Thomas E. Langan. Deep convolutional neural network for flood extent mapping using unmanned aerial vehicles data. *Sensors*, 19(7):1486, March 2019. doi: 10.3390/s19071486.

[6] J. Teng, A.J. Jakeman, J. Vaze, B.F.W. Croke, D. Dutta, and S. Kim. Flood inundation modelling: A review of methods, recent advances and uncertainty analysis. *Environmental Modelling & Software*, 90:201–216, April 2017. doi: 10.1016/j.envsoft.2017.01.006.

[7] Alena Otto, Niels Agatz, James Campbell, Bruce Golden, and Erwin Pesch. Optimization approaches for civil applications of unmanned aerial vehicles (uavs) or aerial drones: A survey. *Networks*, 72(4):411–458, March 2018. doi: 10.1002/net.21818.

[8] Amina Khan, Sumeet Gupta, and Sachin Kumar Gupta. Emerging uav technology for disaster detection, mitigation, response, and preparedness. *Journal of Field Robotics*, 39(6):905–955, April 2022. doi: 10.1002/rob.22075.

[9] Abderahman Rejeb, Karim Rejeb, Steve Simske, and Horst Treiblmaier. Humanitarian drones: A review and research agenda. *Internet of Things*, 16:100434, December 2021. doi: 10.1016/j.iot.2021.100434.

[10] Ebtehal Turki Alotaibi, Shahad Saleh Alqefari, and Anis Koubaa. Lsar: Multi-uav collaboration for search and rescue missions. *IEEE Access*, 7:55817–55832, 2019. doi: 10.1109/access.2019.2912306.

[11] Shuiquan Li, Binqing Zhai, Tianjiao Dang, Mingyue Gao, Yashi Pan, and Yuanshun Wang. Urban flood disaster prevention, control, and rescue using edge computing and the resilience concept. *Scientific Reports*, 16(1), December 2025. doi: 10.1038/s41598-025-31599-6.

[12] Daniele Giovanni Gioia, Katharina Glock, and Jens Kahlen. A perspective and review on adaptive mission planning and hazard monitoring during emergencies with autonomous unmanned systems. *Annual Reviews in Control*, 61:101054, 2026. doi: 10.1016/j.arcontrol.2026.101054.

[13] S. K. McFEETERS. The use of the normalized difference water index (ndwi) in the delineation of open water features. *International Journal of Remote Sensing*, 17(7):1425–1432, May 1996. doi: 10.1080/01431169608948714.

[14] Maryam Rahnemoonfar, Tashnim Chowdhury, Argho Sarkar, Debvrat Varshney, Masoud Yari, and Robin Roberson Murphy. Floodnet: A high resolution aerial imagery dataset for post flood scene understanding. *IEEE Access*, 9:89644–89654, 2021. doi: 10.1109/access.2021.3090981.

[15] Georgios Simantiris and Costas Panagiotakis. Unsupervised color-based flood segmentation in uav imagery. *Remote Sensing*, 16(12):2126, June 2024. doi: 10.3390/rs16122126.

[16] N. S. Ibrahim, S. M. Sharun, M. K. Osman, S. B. Mohamed, and S. H. Y. S. Abdullah. The application of uav images in flood detection using image segmentation techniques. *Indonesian Journal of Electrical Engineering and Computer Science*, 23(2):1219, August 2021. doi: 10.11591/ijeecs.v23.i2.pp1219-1226.

[17] Alvy Ray Smith. Color gamut transform pairs. *ACM SIGGRAPH Computer Graphics*, 12(3):12–19, August 1978. doi: 10.1145/965139.807361.

[18] Robert M. Haralick, Stanley R. Sternberg, and Xinhua Zhuang. Image analysis using mathematical morphology. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-9(4):532–550, July 1987. doi: 10.1109/tpami.1987.4767941.

[19] Satoshi Suzuki and KeiichiA be. Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1):32–46, April 1985. doi: 10.1016/0734-189x(85)90016-7.

[20] P.D Bates and A.P.J De Roo. A simple raster-based model for flood inundation simulation. *Journal of Hydrology*, 236(1-2):54–77, September 2000. doi: 10.1016/s0022-1694(00)00278-x.

[21] Bidur Ghimire, Albert S. Chen, Michele Guidolin, Edward C. Keedwell, Slobodan Djordjević, and Dragan A. Savić. Formulation of a fast 2d urban pluvial flood model using a cellular automata approach. *Journal of Hydroinformatics*, 15(3):676–686, December 2012. doi: 10.2166/hydro.2012.245.

[22] Michele Guidolin, Albert S. Chen, Bidur Ghimire, Edward C. Keedwell, Slobodan Djordjević, and Dragan A. Savić. A weighted cellular automata 2d inundation model for rapid flood analysis. *Environmental Modelling & Software*, 84:378–394, October 2016. doi: 10.1016/j.envsoft.2016.07.008.

[23] Behzad Jamali, Peter M. Bach, Luke Cunningham, and Ana Deletic. A cellular automata fast flood evaluation (caffé) model. *Water Resources Research*, 55(6):4936–4953, June 2019. doi: 10.1029/2018wr023679.

[24] Peter Hart, Nils Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/tssc.1968.300136.

[25] A. Stentz. Optimal and efficient path planning for partially-known environments. In *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, pages 3310–3317. IEEE Comput. Soc. Press, 1994. doi: 10.1109/robot.1994.351061.

[26] Sven Koenig and Maxim Likhachev. D* Lite. In *Proc. 18th National Conference on Artificial Intelligence (AAAI-02)*, pages 476–483, 2002. Available: `https://www.ri.cmu.edu/publications/d-lite`.

[27] S. Koenig and M. Likhachev. Fast replanning for navigation in unknown terrain. *IEEE Transactions on Robotics*, 21(3):354–363, June 2005. doi: 10.1109/tro.2004.838026.

[28] Shubhani Aggarwal and Neeraj Kumar. Path planning techniques for unmanned aerial vehicles: A review, solutions, and challenges. *Computer Communications*, 149:270–299, January 2020. doi: 10.1016/j.comcom.2019.10.014.

[29] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, March 1962. doi: 10.1137/0110015.

[30] Daniel J. Rosenkrantz, Richard E. Stearns, and Philip M. Lewis, II. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, September 1977. doi: 10.1137/0206041.

[31] Chase C. Murray and Amanda G. Chu. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies*, 54:86–109, May 2015. doi: 10.1016/j.trc.2015.03.005.

[32] Boualem Rabta, Christian Wankmüller, and Gerald Reiner. A drone fleet model for last-mile distribution in disaster relief operations. *International Journal of Disaster Risk Reduction*, 28:107–112, June 2018. doi: 10.1016/j.ijdrr.2018.02.020.

[33] Xiya Dong, Benhe Gao, and Runjia Liu. Path optimization for multi-vehicle and multi-uav collaborative delivery in flood rescue under road disruptions: A case study of the 2024 guangdong flood disaster. *Drones*, 10(5):322, April 2026. doi: 10.3390/drones10050322.

[34] Timothy Patterson, Sally McClean, Philip Morrow, Gerard Parr, and Chunbo Luo. Timely autonomous identification of uav safe landing zones. *Image and Vision Computing*, 32(9):568–578, September 2014. doi: 10.1016/j.imavis.2014.06.006.

[35] Maryam Asadzadeh Kaljahi, Palaiahnakote Shivakumara, Mohd Yamani Idna Idris, Mohammad Hossein Anisi, Tong Lu, Michael Blumenstein, and Noorzaily Mohamed Noor. An automatic zone detection system for safe landing of uavs. *Expert Systems with Applications*, 122:319–333, May 2019. doi: 10.1016/j.eswa.2019.01.024.

[36] Lorenz Meier, Petri Tanskanen, Lionel Heng, Gim Hee Lee, Friedrich Fraundorfer, and Marc Pollefeys. Pixhawk: A micro aerial vehicle design for autonomous flight using onboard computer vision. *Autonomous Robots*, 33(1-2):21–39, February 2012. doi: 10.1007/s10514-012-9281-4.

[37] Anis Koubaa, Azza Allouch, Maram Alajlan, Yasir Javed, Abdelfettah Belghith, and Mohamed Khalgui. Micro air vehicle link (mavlink) in a nutshell: A survey. *IEEE Access*, 7:87658–87680, 2019. doi: 10.1109/access.2019.2924410.

[38] MAVLink Development Team. File formats: Mission plain-text file format (QGC WPL). MAVLink Developer Guide. [Online]. Available: `https://mavlink.io/en/file_formats/`, 2026. Accessed: Oct. 6, 2026.

[39] MAVLink Development Team. MAVLink common message set (common.xml). [Online]. Available: `https://github.com/mavlink/mavlink/blob/master/message_definitions/v1.0/common.xml`, 2026. Accessed: Oct. 6, 2026.

[40] ArduPilot Dev Team. Copter mission command list. ArduPilot Documentation. [Online]. Available: `https://ardupilot.org/copter/docs/mission-command-list.html`, 2026. Accessed: Oct. 6, 2026.

[41] PX4 Development Team. Missions. PX4 User Guide. [Online]. Available: `https://docs.px4.io/main/en/flying/missions`, 2026. Accessed: Oct. 6, 2026.

[42] John P. Snyder. Map projections: A working manual. Professional Paper 1395, U.S. Geological Survey, 1987. doi: 10.3133/pp1395.

[43] Sarah E. Battersby, Michael P. Finn, E. Lynn Usery, and Kristina H. Yamamoto. Implications of web mercator and its use in online mapping. *Cartographica*, 49(2):85–101, June 2014. doi: 10.3138/carto.49.2.2313.

[44] T. Vincenty. Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations. *Survey Review*, 23(176):88–93, April 1975. doi: 10.1179/sre.1975.23.176.88.

[45] Charles F. F. Karney. Algorithms for geodesics. *Journal of Geodesy*, 87(1):43–55, June 2012. doi: 10.1007/s00190-012-0578-z.

[46] G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000. No DOI; standard citation requested by the OpenCV project.

[47] Patrick Zippenfenig. Open-meteo.com weather api. Zenodo, 2024. doi: 10.5281/ZENODO.7970649.

[48] Open-Meteo. Weather forecast API documentation. [Online]. Available: `https://open-meteo.com/en/docs`, 2026. Accessed: Oct. 6, 2026.

[49] MAVLink Development Team. Mission protocol. MAVLink Developer Guide. [Online]. Available: `https://mavlink.io/en/services/mission.html`, 2026. Accessed: Oct. 6, 2026.